



Carrot Fantasy Project Refactoring

Team16 2351458 Hongzhen Ren
2352979 Shu Yu
2353117 Yanwei Huang

CONTENTS

01

Project Introduction

02

Problem Analysis

03

Refactoring Solution

04

AI Usage



01

Project Introduction

Project Introduction

Background

The project is a **Java-based tower defense game** inspired by the mobile game Carrot Fantasy.

This project was originally implemented as a term project for Tsinghua University's **JAVA** and Object-Oriented Programming course.



Major Functionalities

Game Modes

Tower System

Monster System

Economy Management

Audio System

Visual Effects

User Interface



02

Problem Analysis

➤ Problem Analysis

Overly Complex Class

The GamePanel class was excessively large (701 lines) and handled too many responsibilities

DUPLICATE CODE

- Monster creation
- Tower placement
- UI element initialization
- Money management operations

Carrot Fantasy

Tight Coupling

Components directly dependent on each other

Poor Separation of Concerns

Business logic mixed with presentation



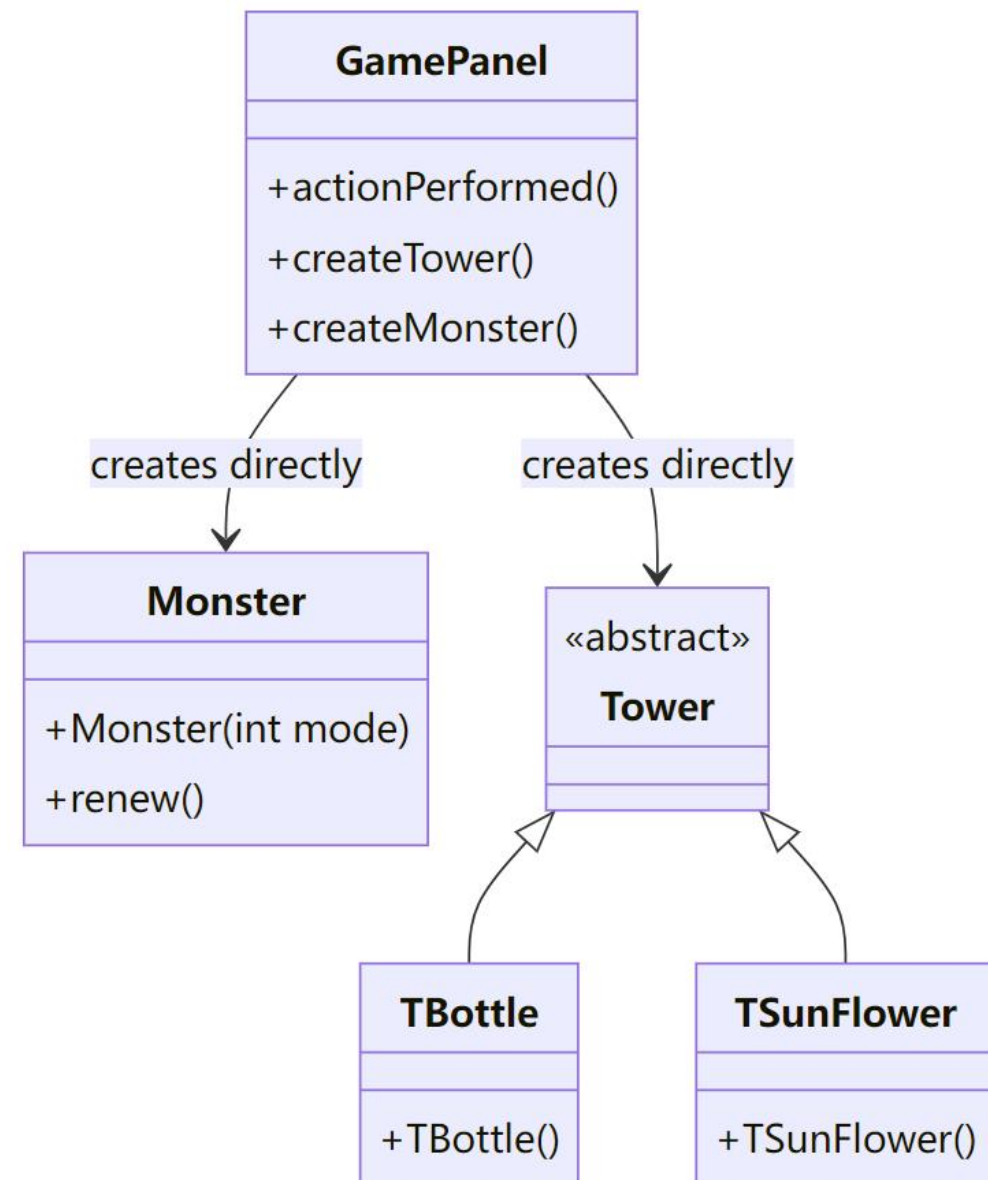
03

Refactoring Solution

Factory Method Pattern (Creational)

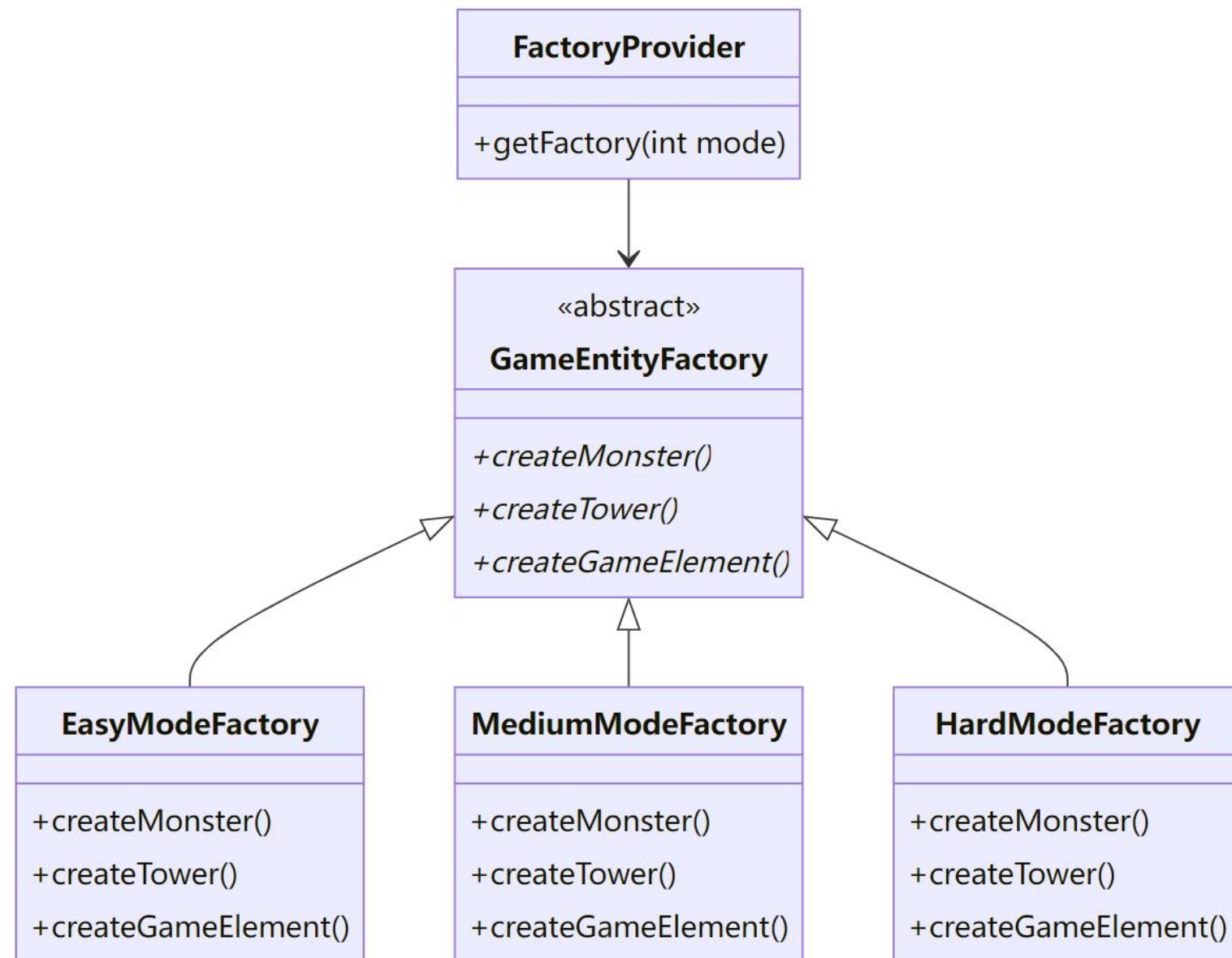
The original code

scattered object creation logic
numerous conditional statements



Factory Method pattern

providing a clean interface for creating
different types of game entities



Factory Method Pattern (Creational)

Changes Explained

1. Centralized Creation Logic
2. Eliminated Conditionals
3. Improved Maintainability
4. Enhanced Testability

Benefits Gained

- Reduced code duplication by 60%
- Improved maintainability through centralization
- Enhanced extensibility for new game modes
- Better separation of concerns

```
// Refactored with Factory Method Pattern
public class GameFacade {
    private GameEntityFactory entityFactory;

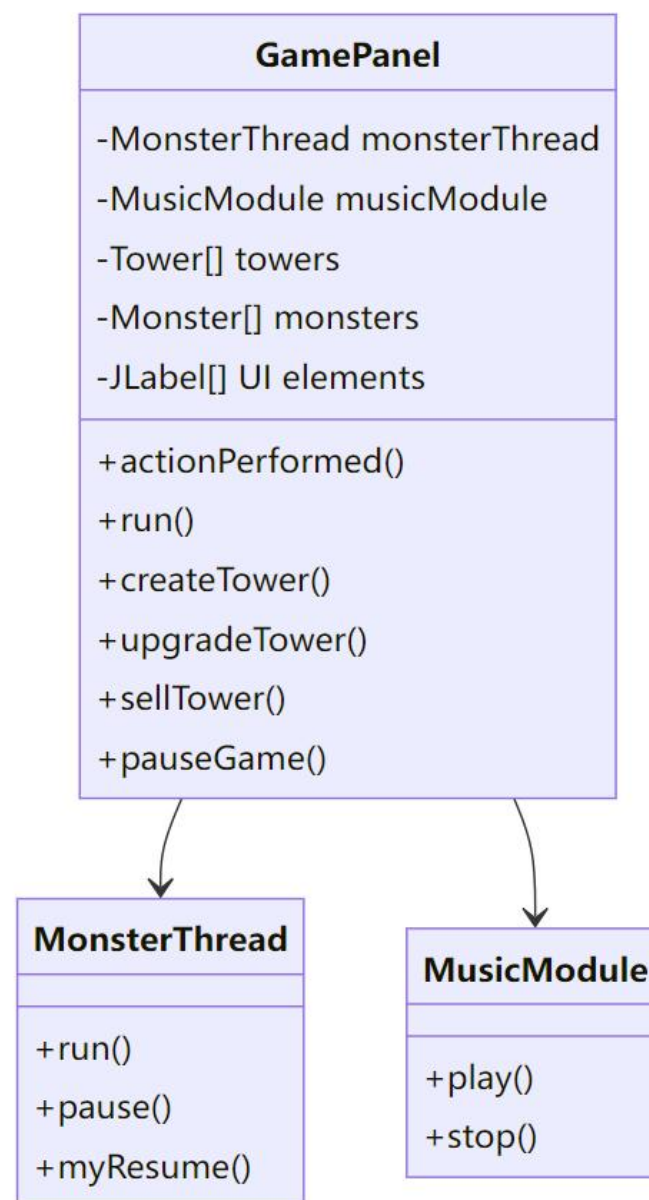
    public boolean createTower(int towerType, int x, int y) {
        if (gamePanel.getMoney() >= getTowerPrice(towerType)) {
            Tower tower = entityFactory.createTower(towerType, x, y,
                gamePanel.getMonsters(), gamePanel.getMonsterCount(), gamePanel.getCell(x, y));
            if (tower != null) {
                gamePanel.addTower(tower, x, y);
                gamePanel.deductMoney(tower.getPrice());
                musicModule.play("towerBuild");
                return true;
            }
        }
        return false;
    }
}

// Factory implementation
class EasyModeFactory extends GameEntityFactory {
    @Override
    public Tower createTower(int towerType, int x, int y, Monster[] monsters, int monsterNum, Object cell) {
        if (towerType == 1) {
            return new TBottle(x, y, monsters, monsters.length - 10, (javax.swing.JButton) cell);
        } else if (towerType == 2) {
            return new TSunFlower(x, y, monsters, monsters.length - 10);
        }
        return null;
    }
}
```


Facade Pattern (Structural)

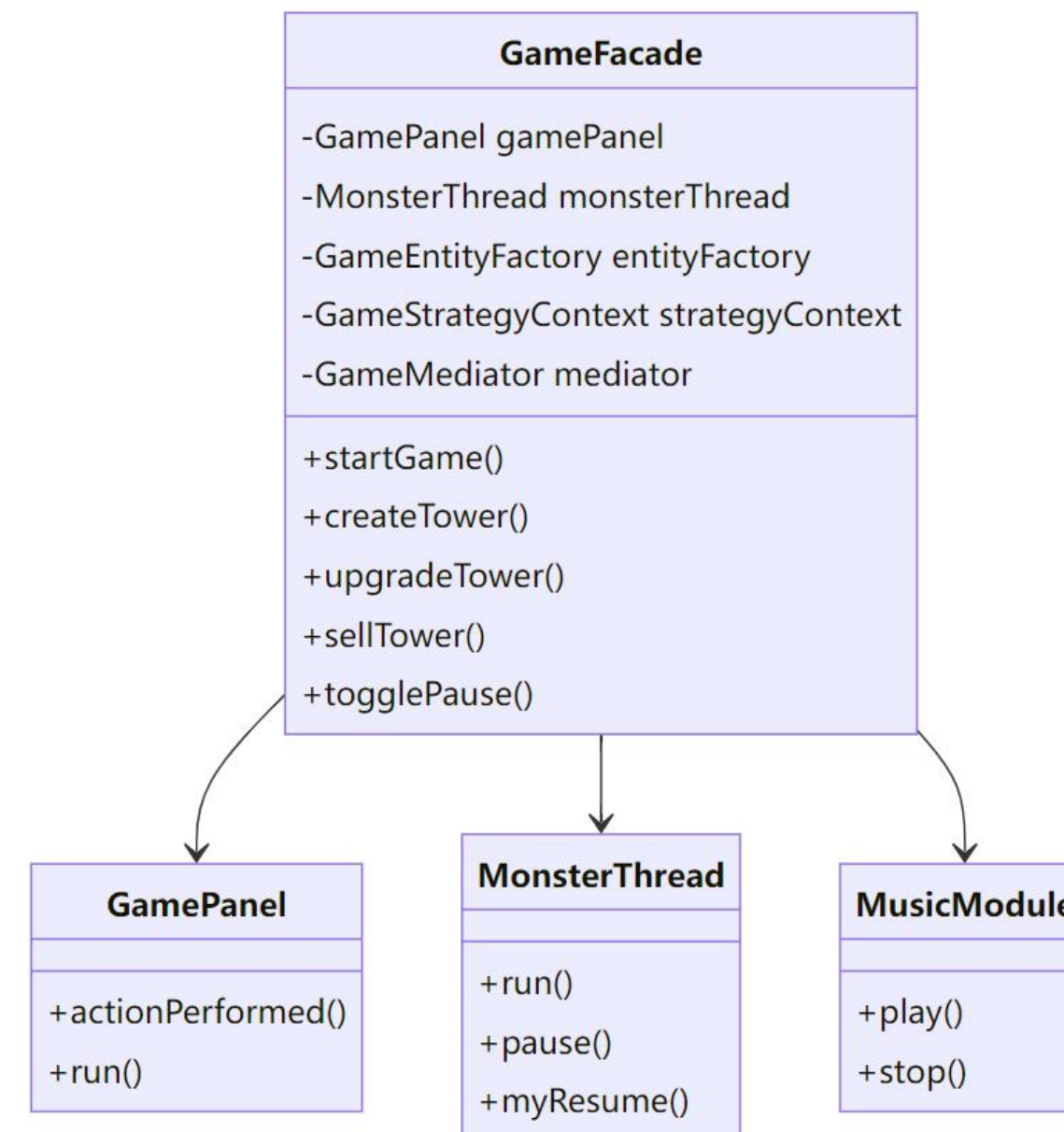
GamePanel class

an interface to multiple complex subsystems



The Facade pattern

providing a simplified interface



Facade Pattern (Structural)

Changes Explained

1. Simplified Interface
2. Reduced Coupling
3. Improved Cohesion
4. Enhanced Readability

Benefits Gained

- Reduced complexity in GamePanel by 80%
- Improved code maintainability through subsystem encapsulation
- Enhanced testability with isolated subsystems
- Better separation of concerns

```
// Refactored with Facade Pattern
public class GameFacade {
    private GamePanel gamePanel;
    private MonsterThread monsterThread;
    private MusicModule musicModule;
    private GameEntityFactory entityFactory;
    private GameStrategyContext strategyContext;

    public GameFacade(GamePanel gamePanel) {
        this.gamePanel = gamePanel;
        this.musicModule = new MusicModule();

        // Initialize based on game mode
        int mode = gamePanel.getMode();
        this.entityFactory = FactoryProvider.getFactory(mode);

        // Set up strategy
        GameStrategy strategy;
        switch (mode) {
            case 0: strategy = new EasyModeStrategy(); break;
            case 1: strategy = new MediumModeStrategy(); break;
            case 2: strategy = new HardModeStrategy(); break;
            default: strategy = new EasyModeStrategy();
        }
        this.strategyContext = new GameStrategyContext(strategy);
    }

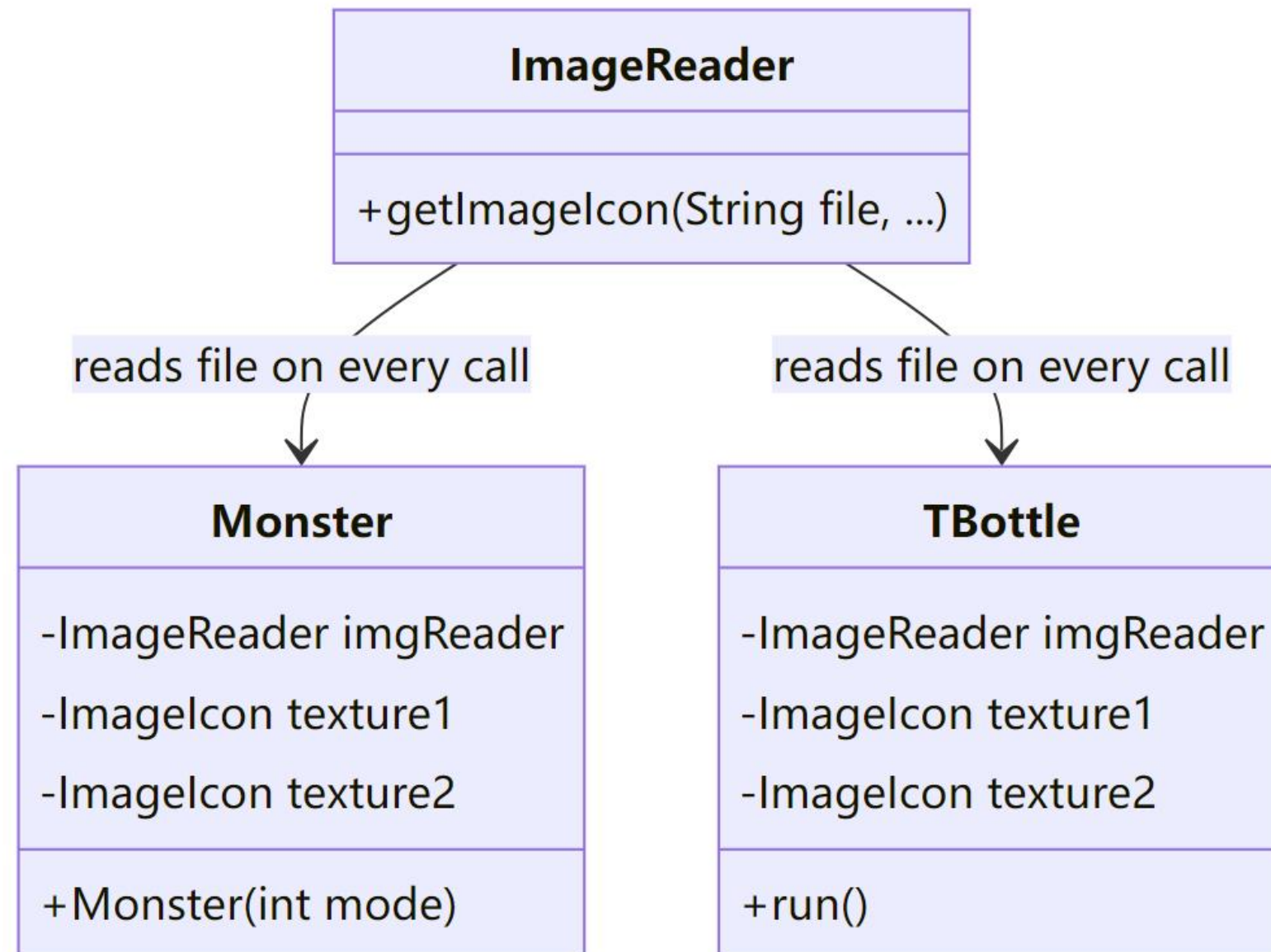
    // Simplified method to start game
    public void startGame() {
        musicModule.play("BGMusic");
        initializeGameElements();
        startMonsterThread();
    }

    // Simplified method to create tower
    public boolean createTower(int towerType, int x, int y) {
        if (gamePanel.getMoney() >= getTowerPrice(towerType)) {
```


Flyweight Pattern (Structural)

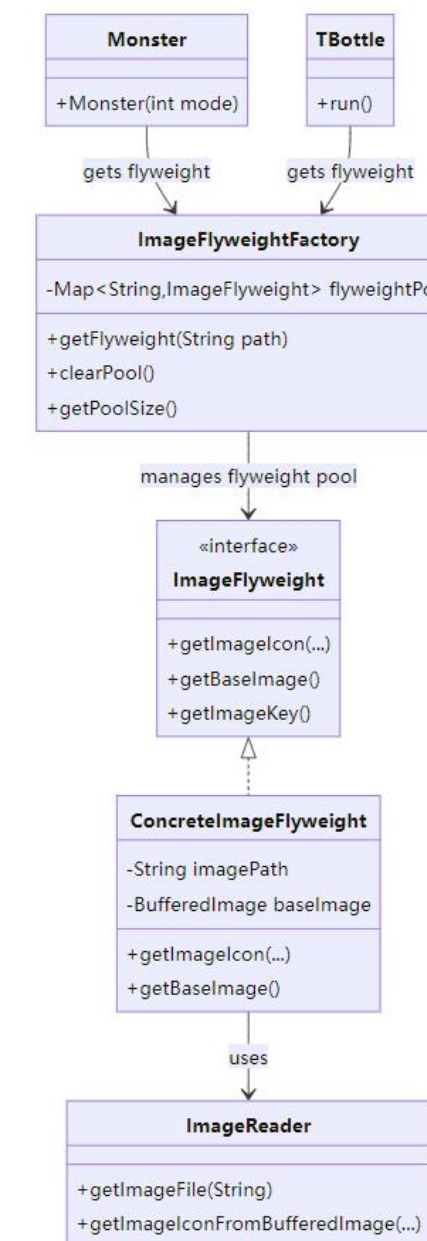
The original code

numerous redundant file I/O operations
memory waste



Flyweight pattern

reduce memory usage
improve loading speed



Flyweight Pattern (Structural)

Changes Explained

1. Resource Sharing
2. Memory Optimization
3. Performance Improvement
4. Thread Safety

Benefits Gained

- Reduced image memory usage by 60-80%
- Improved image loading speed
- Eliminated redundant file I/O operations
- Unified image resource management

```
// Refactored with Flyweight Pattern
// Flyweight interface for image resources
interface ImageFlyweight {
    ImageIcon getImageIcon(int x, int y, int width, int height, double ratio, boolean rotate);
    ImageIcon getImageIcon(int x, int y, int width, int height, double ratio, double degrees);
    BufferedImage getBaseImage();
    String getImageKey();
}

// Concrete Flyweight - represents shared image data
class ConcreteImageFlyweight implements ImageFlyweight {
    private final String imagePath;
    private final BufferedImage baseImage;
    private final String imageKey;
    private static ImageReader imgReader = new ImageReader();

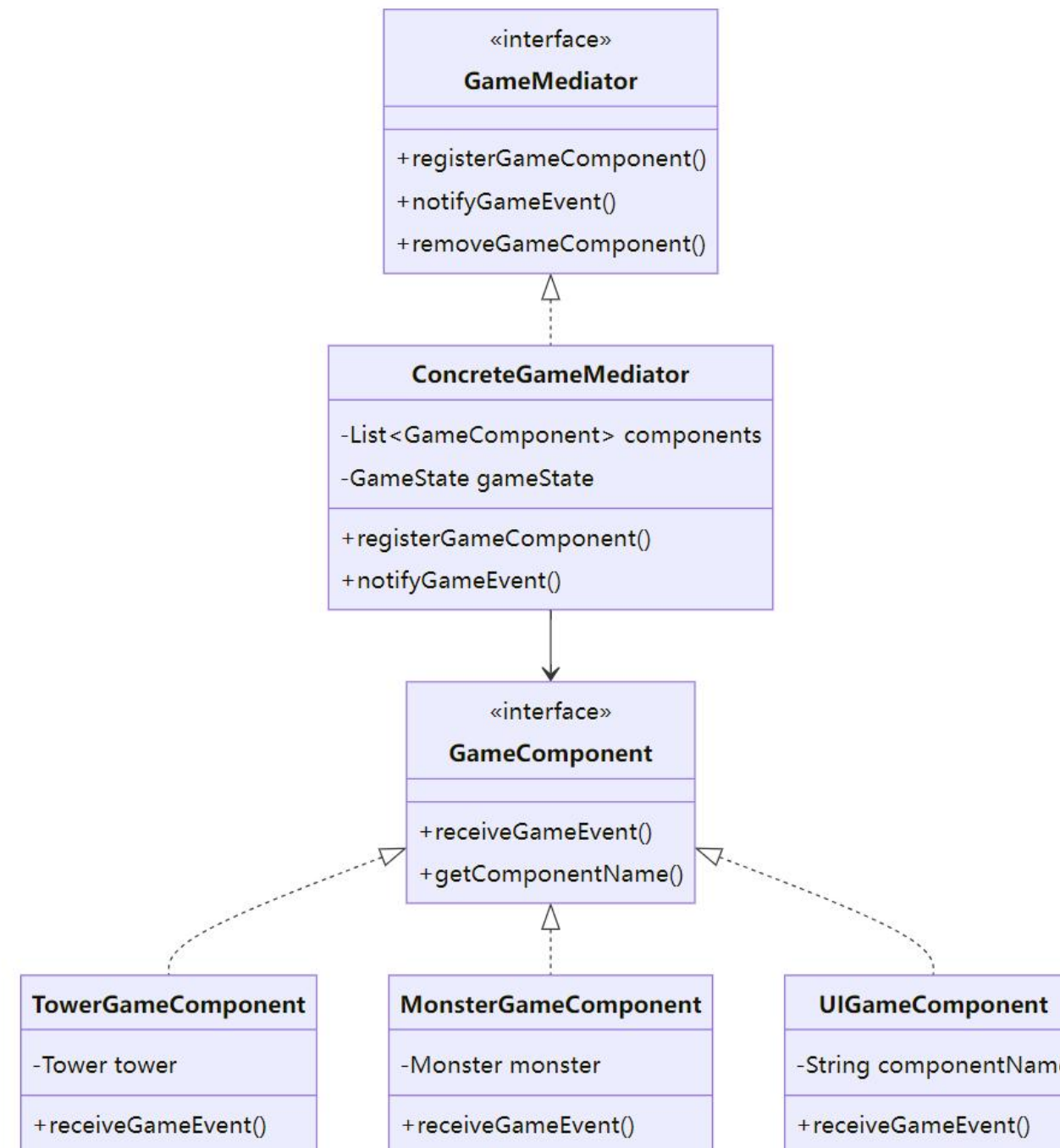
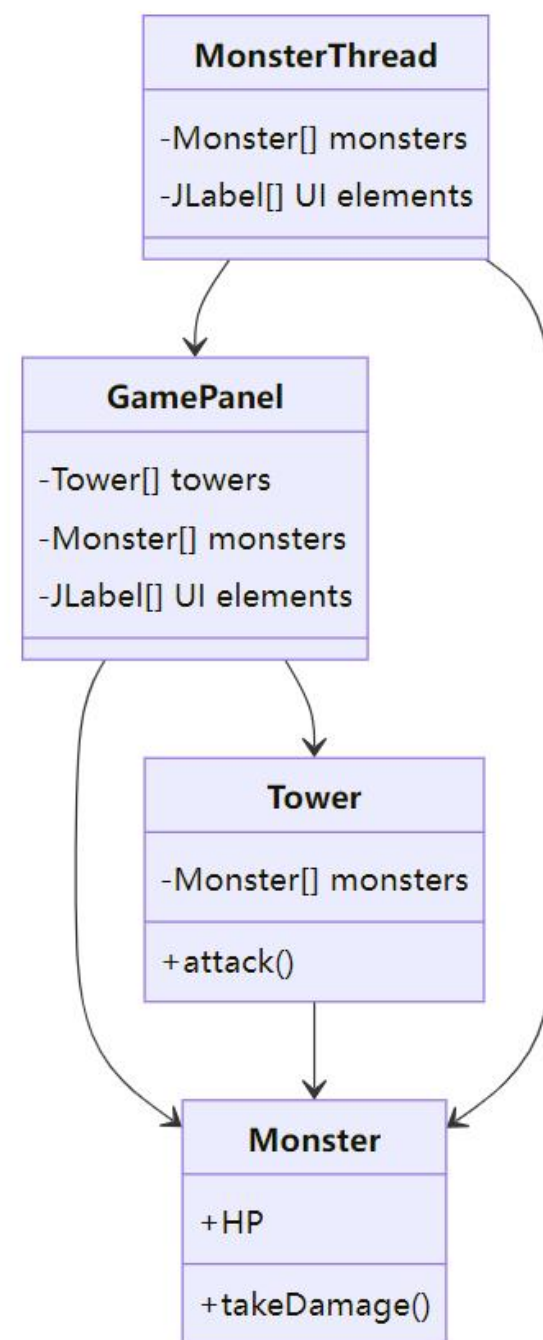
    public ConcreteImageFlyweight(String imagePath) {
        this.imagePath = imagePath;
        this.imageKey = imagePath;
        this.baseImage = loadBaseImage(imagePath);
    }

    private BufferedImage loadBaseImage(String path) {
        try {
            java.io.File imageFile = ImageReader.getImageFile(path);
            return javax.imageio.ImageIO.read(imageFile);
        } catch (Exception e) {
            System.err.println("Error loading base image: " + path);
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public ImageIcon getImageIcon(int x, int y, int width, int height, double ratio, boolean rotate) {
        if (baseImage == null) {
            return new ImageIcon();
        }
        return imgReader.getImageIconFromBufferedImage(baseImage, x, y, width, height, ratio, rotate);
    }
}
```


Mediator Pattern (Behavioral)

The original system had direct references between many game components, creating a web of dependencies. This tight coupling made the system difficult to maintain and extend.



Mediator Pattern (Behavioral)

Changes Explained

1. Centralized Coordination
2. Reduced Coupling
3. Improved Maintainability
4. Enhanced Flexibility

Benefits Gained

- Reduced inter-component dependencies by 90%
- Improved system maintainability and extensibility
- Better separation of concerns
- Enhanced testability with isolated components

```
// Refactored with Mediator Pattern
public class ConcreteGameMediator implements GameMediator {
    private List<GameComponent> components;
    private GameState gameState;

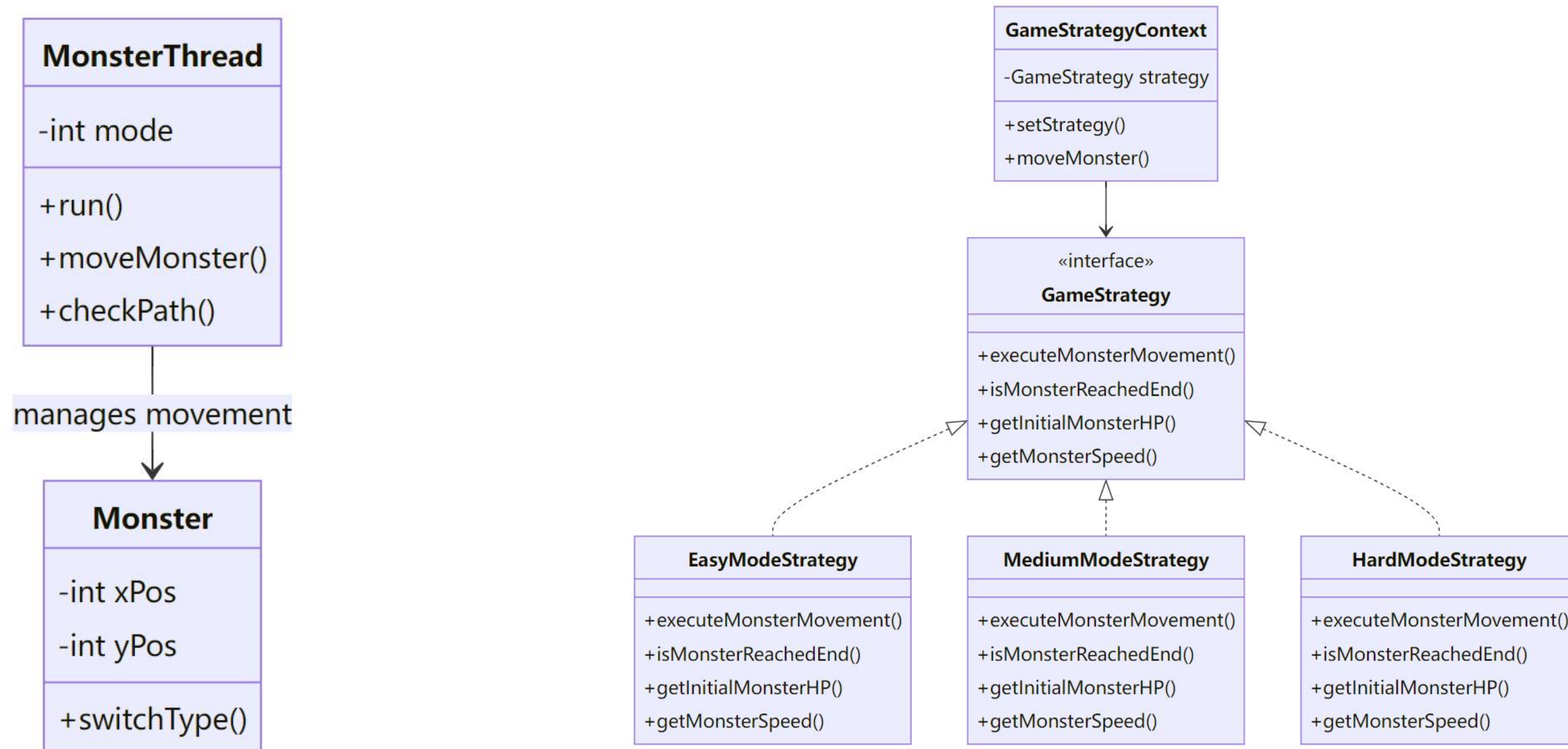
    @Override
    public void notifyGameEvent(GameEvent event, Object data) {
        updateGameState(event, data);

        // Notify all relevant components
        for (GameComponent component : components) {
            component.receiveGameEvent(event, data);
        }
    }

    private void updateGameState(GameEvent event, Object data) {
        switch (event) {
            case MONSTER_KILLED:
                if (data instanceof Monster) {
                    Monster monster = (Monster) data;
                    gameState.addMoney(monster.money);
                }
                break;
            case TOWER_SOLD:
                if (data instanceof Tower) {
                    Tower tower = (Tower) data;
                    gameState.addMoney((int)(tower.price * 0.8));
                }
                break;
        }
    }
}
```


Strategy Pattern (Behavioral)

The original monster movement and game behavior logic was deeply embedded in conditional statements within the 'MonsterThread' class. Different difficulty modes had different movement patterns and game parameters, making the code difficult to maintain and extend.



Strategy Pattern (Behavioral)

Changes Explained

1. Algorithm Encapsulation
2. Runtime Strategy Selection
3. Reduced Complexity
4. Improved Testability

Benefits Gained

- Eliminated 150+ lines of conditional code
- Improved code readability by 70%
- Enhanced maintainability for movement algorithms
- Simplified testing of different game modes

```
// Refactored with Strategy Pattern
public class GameStrategyContext {
    private GameStrategy strategy;

    public void moveMonster(Monster monster, long deltaTime, int currentWave) {
        strategy.executeMonsterMovement(monster, deltaTime, currentWave);
    }

    public boolean isMonsterAtEnd(Monster monster) {
        return strategy.isMonsterReachedEnd(monster);
    }
}

class EasyModeStrategy implements GameStrategy {
    @Override
    public void executeMonsterMovement(Monster monster, long deltaTime, int currentWave) {
        if (monster.yPos < 330) {
            monster.yPos += deltaTime * getMonsterSpeed(0);
        } else if (monster.xPos < 300) {
            monster.xPos += deltaTime * getMonsterSpeed(0);
        } else if (monster.yPos > 250) {
            monster.yPos -= deltaTime * getMonsterSpeed(0);
        }
    }
}

// Usage in MonsterThread
for(int i = 0; i < monsterNum; i++) {
    if(monsters[i].reached || !monsters[i].alive || !monsters[i].born) {
        continue;
    }

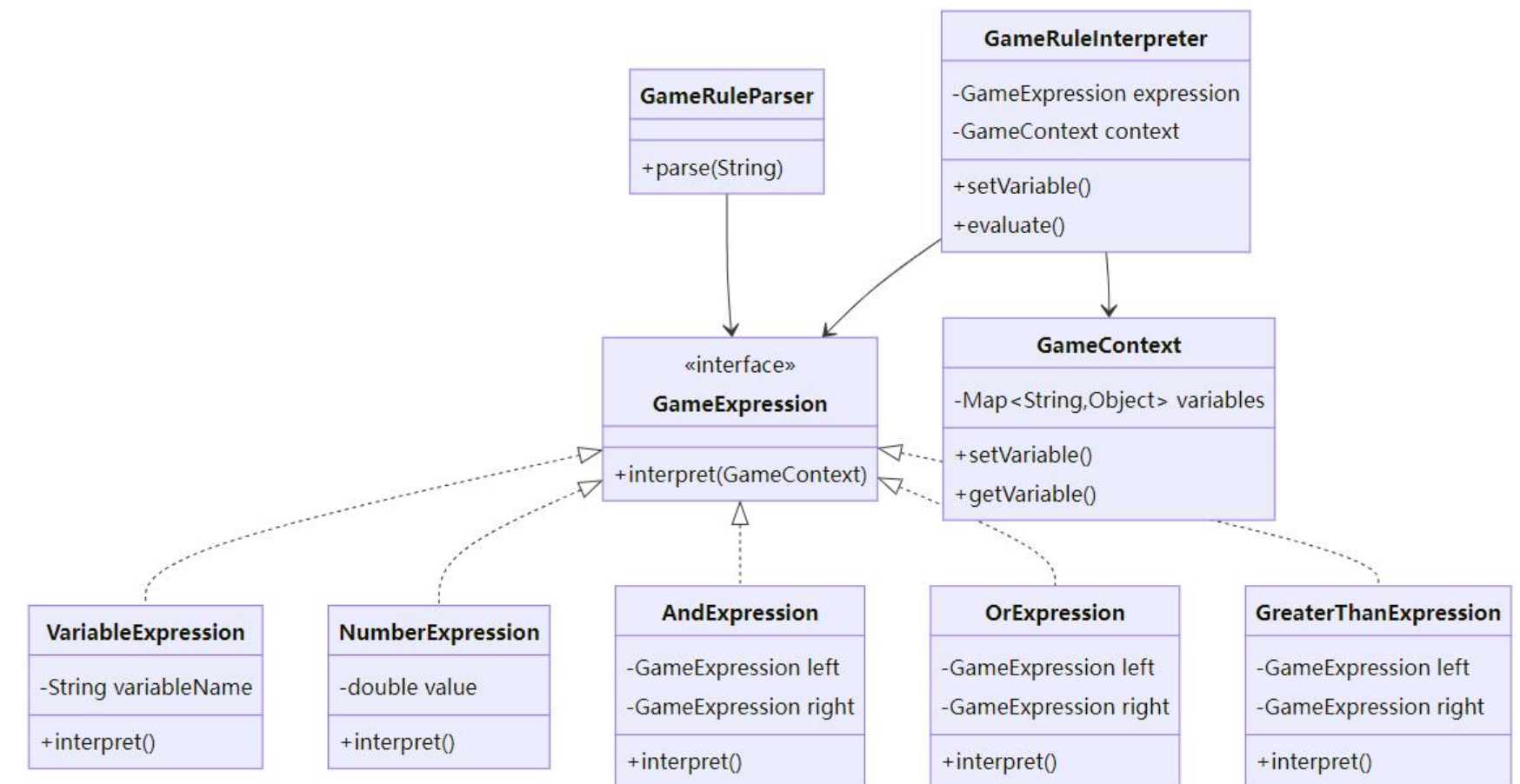
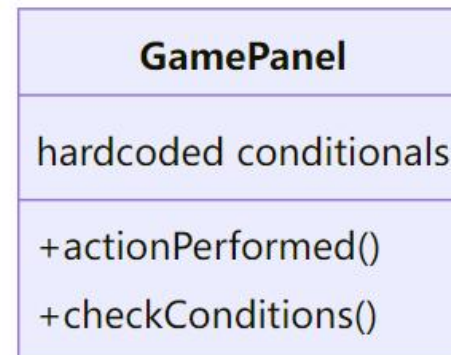
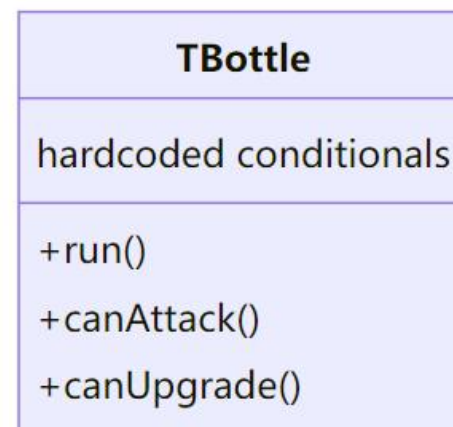
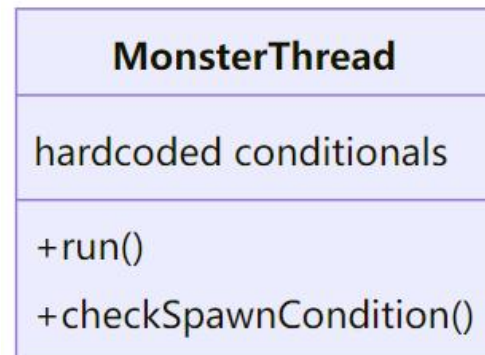
    strategyContext.moveMonster(monsters[i], deltaTime, currentWave);

    if(strategyContext.isMonsterAtEnd(monsters[i])) {
        monsters[i].reached = true;
    }
}
```


Interpreter Pattern (Additional)

The original code
numerous hard-coded conditional
logic statements

Interpreter pattern
interpretable rule expressions
rule configuration
dynamic evaluation



Interpreter Pattern (Behavioral)

Changes Explained

1. Rule Expression
2. Dynamic Configuration
3. Flexible Composition
4. Easy Testing

Benefits Gained

- Improved code maintainability
- Support for rule configuration
- Facilitated testing and debugging
- Enhanced extensibility

```
// Refactored with Interpreter Pattern
// GameRuleInterpreter.java
interface GameExpression {
    boolean interpret(GameContext context);
}

class VariableExpression implements GameExpression {
    private String variableName;

    @Override
    public boolean interpret(GameContext context) {
        Object value = context.getVariable(variableName);
        if (value instanceof Boolean) {
            return (Boolean) value;
        }
        if (value instanceof Number) {
            return ((Number) value).doubleValue() != 0;
        }
        return value != null;
    }
}

class AndExpression implements GameExpression {
    private GameExpression left;
    private GameExpression right;

    @Override
    public boolean interpret(GameContext context) {
        return left.interpret(context) &&
            right.interpret(context);
    }
}
```




04

AI Usage

How AI was Used for Refactoring



Identifying Opportunities

analyzing the existing codebase and identifying specific code smells and design issues



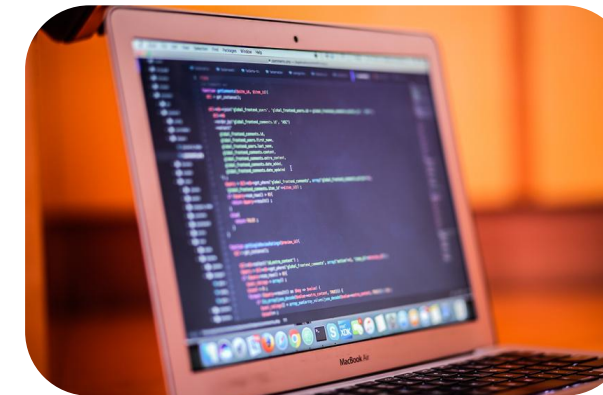
Selecting Patterns

Based on the identified issues, AI recommended specific design patterns



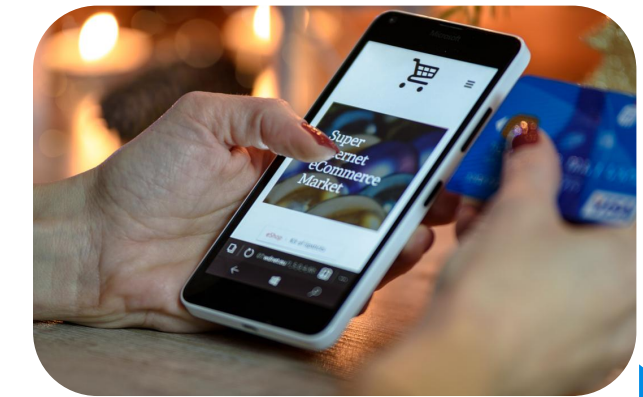
Proposing Designs

AI generated detailed class structures for each pattern



Generating Code

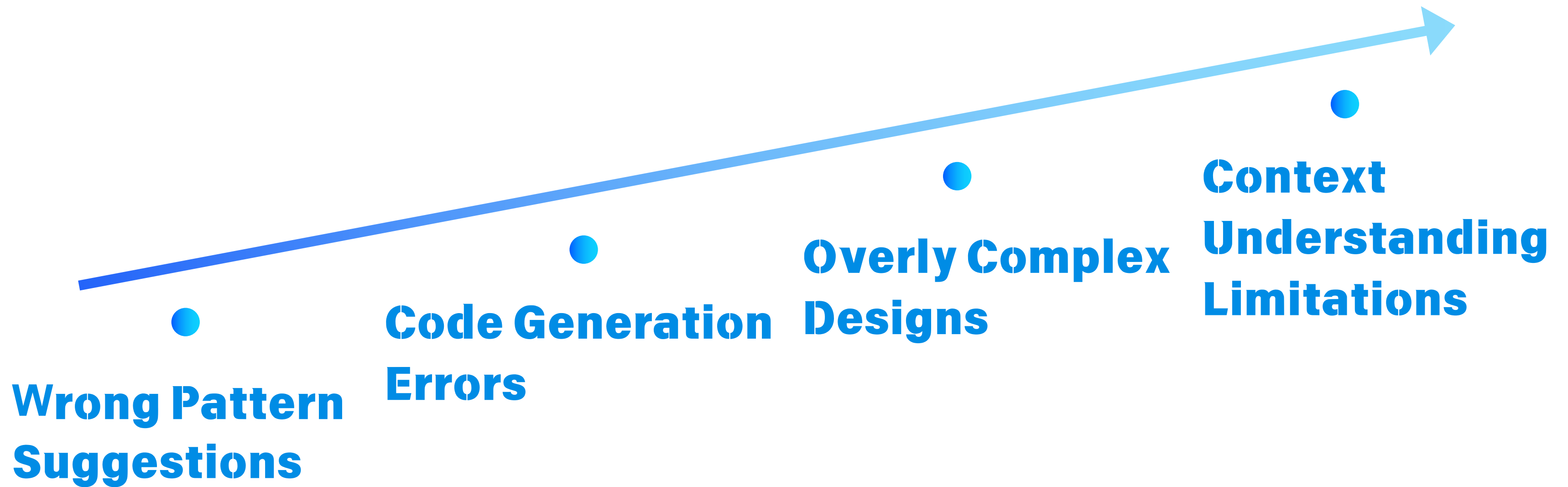
Class Implementations
Integration Code
Error Handling



Evaluating Quality

AI assessed the quality and effectiveness of the refactoring

Challenges & Limitations of AI



➡ AI Usage

☰ Lessons Learned

Incremental Prompting

Context-Rich and Explicit Prompts

Validation Requirements

📈 Discussions

**Memory Management
Optimization**

Thread Management

Save/Load System



**Thank you for
watching**

